

《FastAPI学生管理系统》实战项目作业布置与验收标准

作为实战AI编程培训讲师，结合FastAPI、MySQL、RESTful API规范及团队协作要求，为你定制本次实战作业方案，明确作业内容、开发流程、验收标准及答辩模拟环节，助力学生扎实掌握技术栈并具备项目实战能力。

一、作业核心目标

1. 让学生熟练掌握**FastAPI框架核心用法**（路由设计、参数校验、Pydantic模型、异常处理）；
2. 巩固**MySQL数据库设计**、CRUD操作、关联查询、统计分析的实操能力；
3. 培养团队协作能力、接口文档规范意识及项目全流程开发思维；
4. 衔接AI编程实战场景，为后续AI智能体、知识库扩展打下基础。

二、作业核心任务（学生需完成内容）

（一）团队分组与分工

1. 以**6-7人**为一组，组长负责统筹、进度跟进、答辩安排；
2. 小组内明确分工，建议参考以下角色分配（可调整）：
 - 数据库设计岗：负责ER图绘制、表结构设计、索引优化、关联关系定义；
 - 后端开发岗：负责FastAPI接口开发、业务逻辑实现、数据校验、异常处理；
 - 接口文档岗：负责Swagger文档完善、接口注释编写、接口测试验证；
 - 统计分析岗：负责统计分析模块接口开发、复杂SQL编写、结果验证；
 - 测试与调试岗：负责接口功能测试、边界场景排查、代码bug修复。

（二）核心开发任务

1. 数据库设计与实现

- 基于需求文档，设计**合理的表结构**，需包含以下核心表（含冗余字段设计说明）：
 - student**（学生基本信息表）：包含学生编号、班级、姓名、籍贯、毕业院校、专业、入学/毕业时间、学历、顾问编号、年龄、性别等字段；
 - class**（班级信息表）：班级编号、开课时间、班主任、授课老师等字段；
 - teacher**（老师信息表）：老师基本信息、带班信息等字段；
 - score**（学生考核成绩表）：学生编号、考核序次、成绩等字段；
 - employment**（学生就业信息表）：学生编号、就业开放时间、offer下发时间、公司名称、薪资等字段（含冗余字段：学生姓名、班级）；
- 绘制**ER实体关系图**，明确表之间关联关系（如学生表与班级表、成绩表的关联）；
- 编写**MySQL建表SQL语句**，包含字段约束（主键、外键、非空、唯一）、索引设计、注释；
- 完成**基础数据初始化**：插入至少5条班级数据、20条学生数据、15条老师数据、30条成绩数据、10条就业数据，用于后续接口测试。

2. FastAPI后端系统开发

- 搭建项目工程结构，遵循模块化设计，推荐结构如下：

```
student_management_system/
├── main.py                # 项目入口（FastAPI实例创建、路由注册）
├── database.py            # 数据库连接配置（MySQL连接、会话管理）
├── models/                # 数据库模型（SQLAlchemy ORM定义）
│   ├── __init__.py
│   ├── student.py
│   ├── class_.py
│   ├── teacher.py
│   ├── score.py
│   └── employment.py
├── schemas/               # Pydantic数据模型（请求/响应校验）
│   ├── __init__.py
│   ├── student.py
│   ├── score.py
│   ├── employment.py
│   └── common.py
├── api/                   # 接口路由模块
│   ├── __init__.py
│   ├── student.py
│   ├── score.py
│   ├── employment.py
│   ├── class_.py
│   ├── teacher.py
│   ├── teacher.py
│   ├── statistics.py
│   └── text2sql.py # 额外
├── crud/                  # 数据操作层（CRUD逻辑封装）
│   ├── __init__.py
│   ├── student.py
│   ├── score.py
│   ├── employment.py
│   └── statistics.py
├── utils/                 # 工具模块（异常处理、通用函数、日志）
│   ├── __init__.py
│   ├── exception.py
│   └── common.py
└── requirements.txt       # 项目依赖包列表
```

- 核心接口开发（严格遵循RESTful规范，覆盖需求中所有功能）：
 1. 学生基本信息管理模块：实现 `/students` 相关GET/POST/PUT接口，支持按编号、姓名、班级筛选查询、逻辑删除；
 2. 成绩管理模块：实现成绩录入、修改、删除、按学生ID查询接口，支持考核成绩的增删改查；
 3. 就业管理模块：实现学生/班级就业信息查询、添加/更新接口，支持按编号、公司、薪资范围筛选；
 4. 班级/老师管理模块：实现班级、老师信息的增删改查接口；
 5. 统计分析模块：完成所有统计需求接口，具体如下面表格；
 6. AI模块实现智能生成sql或者智能问数；

统计类型	接口功能
基本信息统计	1. 查询30岁以上学员列表；2. 统计各班级人数、男女生人数
成绩统计	1. 查询每次考试均80分以上的学生信息；2. 查询2次以上不及格的学生信息；3. 统计各班级平均分（降序）
就业统计	1. 统计就业薪资前五的学生信息；2. 计算单个学生就业时长；3. 统计各班级平均就业时长（仅统计就业阶段学生）

- 技术要求：
 - 1. 采用SQLAlchemy ORM实现数据库操作，禁止直接拼接原生SQL（复杂统计场景可结合原生SQL优化）；
 - 2. 所有接口参数通过Pydantic模型完成校验，返回统一的响应格式（含状态码、消息、数据）；
 - 3. 处理异常场景：如查询不存在的学生ID、录入重复数据、参数格式错误等，返回友好的异常提示；
 - 4. 自动生成Swagger API文档（FastAPI自带），确保接口注释清晰、参数说明完整。

3. 接口测试与文档完善

- 借助Swagger文档或Postman，对所有接口进行**功能测试**，验证接口返回结果与需求一致；
- 编写**接口测试用例**（至少覆盖80%核心接口），包含正常场景、异常场景测试；
- 完善Swagger文档接口描述，补充接口示例、返回值说明、参数约束。

4. 后续扩展准备（非强制开发）

- 提前设计**用户认证模块**方案：基于JWT实现管理员、老师、学生不同角色的权限控制，明确接口访问权限；
- 规划**AI智能体集成思路**：说明如何结合大模型实现学生信息智能查询、就业推荐、成绩分析等功能，撰写1页纸方案。

（三）团队协作与文档输出

1. 小组内完成**需求讨论纪要**：明确最终确定的表结构、接口设计细节、技术选型细节；
2. 编写**项目开发文档**，包含以下内容：
 - 项目概述、技术栈、开发环境配置；
 - 数据库设计（ER图、表结构说明、建表SQL）；
 - 接口文档（Swagger文档导出、接口列表、参数说明）；
 - 开发流程记录、分工明细、进度规划；
 - 扩展模块设计方案（用户认证、AI智能体）。

三、开发流程要求（学生需严格执行）

1. **需求分析**：小组全员研读需求文档，拆解核心功能，确认表结构设计思路、接口命名规范；
2. **数据库设计**：完成ER图绘制、建表SQL编写，组内交叉审核，确保表结构合理、关联关系清晰；
3. **系统概要设计**：确定项目工程结构、核心模块划分、接口路由设计、响应格式规范；
4. **功能开发**：按模块分工开发，每日同步进度，解决开发过程中的技术问题；
5. **测试**：完成接口测试、bug修复，确保所有功能正常运行，文档完整。

四、小组团体验收标准

(一) 基础分 (60分)

1. 项目工程结构规范 (10分)：符合模块化设计，目录清晰、文件命名合理，无冗余文件；
2. 数据库设计 (15分)：表结构符合需求、字段约束/索引设计合理、建表SQL可直接执行；
3. 核心接口开发 (20分)：覆盖需求中所有基础功能接口，RESTful规范达标，参数校验有效，无核心功能缺失；
4. 统计分析模块 (10分)：所有统计接口功能正常，统计结果准确，复杂SQL编写规范；
5. 文档完整性 (5分)：需求讨论纪要、开发文档、Swagger文档完整，无关键信息缺失。

(二) 加分项 (40分)

1. 代码质量 (10分)：代码注释清晰、逻辑简洁、符合Python编码规范，无冗余代码，异常处理完善；
2. 技术深度 (20分)：
 1. 有数据库E-R图、项目设计文档、清晰明了专业。
 2. 采用**数据库连接池**优化MySQL连接性能，配置合理的 `pool_size` 等参数；
 3. 结合**日志模块** (如 `logging/loguru`) 实现接口操作日志、异常日志记录；
 4. 对超大数据查询场景 (如统计分析)，采用**分页查询**、**多线程/协程**、**生成器**等技术优化性能；
 5. 完成**用户认证模块**基础开发 (JWT权限控制)，实现角色权限区分；
 6. 完成注册账号时**密码MD5加密后存储到数据库**，**登录时用MD5进行校验是否正确**。
 7. 其他上述没写到，但是**非常具有自己扩展的技术或者想法思路创新的点**。
3. 扩展性 (10分)：代码模块化程度高，便于后续新增功能 (如AI智能体、顾问管理模块) 集成，扩展方案设计合理；
4. 团队协作 (5分)：分工明确、进度同步顺畅，答辩时小组成员配合默契，无推诿情况。

(三) 扣分项

1. 核心功能缺失：如统计分析模块接口未完成、基础CRUD接口存在严重bug，扣10-20分；
2. 代码运行报错：项目无法正常启动、接口调用报错，扣5-15分；
3. 文档缺失/不完整：无开发文档、Swagger文档无法访问，扣5-10分；
4. 团队协作混乱：答辩时成员无法回答基础问题、分工不明确，扣5分。

五、个人答辩模拟实战验收环节

(一) 答辩基本要求

1. 每组答辩时长**15-20分钟**，其中：
 - 需求讲解+方案设计：5分钟；
 - 代码实现演示：8-10分钟；
 - 问题回答：2-5分钟。
2. 答辩人员安排：每组安排6-7人，分别负责方案讲解、代码演示、核心问题回答，**需抽查小组内所有成员**参与回答问题。

(二) 答辩验收核心环节

1. 需求讲解与方案设计验收 (20分)

- 考察点：
 1. 小组是否准确理解需求，核心功能拆解是否完整；
 2. 数据库设计思路：为何选择该表结构、关联关系如何定义、冗余字段设计原因；
 3. 接口设计思路：RESTful规范的体现、参数校验的设计逻辑、统计模块接口的设计考量；
 4. 技术选型理由：为何选择FastAPI、SQLAlchemy，相比其他框架的优势。
- 评分标准：讲解清晰、逻辑通顺、设计合理得20分；讲解模糊、设计存在明显缺陷扣5-15分。

2. 代码实现演示验收 (40分)

- 考察点：
 1. 项目启动与运行：演示项目正常启动，Swagger文档可正常访问；
 2. 核心接口演示：
 - 基础CRUD接口：演示学生/成绩/就业信息的增删改查，验证数据持久化效果；
 - 统计分析接口：重点演示3类统计场景，验证统计结果准确性（可结合初始化数据对比）；
 - 异常处理：故意传入错误参数，验证接口异常提示是否友好；
 3. 代码亮点展示：演示数据库连接池优化、日志记录、性能优化等技术点的实现代码；
 4. 扩展模块展示：演示用户认证模块（若开发完成）或AI智能体集成方案。
- 评分标准：演示流畅、代码逻辑清晰、功能正常得40分；演示卡顿、功能存在问题扣5-20分。

3. 问题回答验收 (30分)

- 问题分类与占比：
 1. 本次项目相关问题（80%）：围绕表结构设计、接口开发逻辑、SQL语句编写、代码调试思路提问；
 2. 前置知识相关问题（20%）：考察Python基础、SQLAlchemy原理、FastAPI核心概念、Pydantic校验规则等；
- 考察点：
 1. 成员对项目代码的熟悉程度：能否解读核心接口代码、说明业务逻辑；
 2. 技术原理解释：能否解释SQLAlchemy ORM与原生SQL的区别、FastAPI异步与同步的差异；
 3. 问题解决能力：针对开发中遇到的bug（如数据库连接报错、接口参数校验失败），说明排查思路与解决方案；
 4. 扩展设计思路：能否说明用户认证模块、AI智能体的实现逻辑。
- 评分标准：回答准确、思路清晰得30分；回答错误、思路模糊扣5-15分。

4. 团队协作与答辩表现验收 (10分)

- 考察点：
 1. 小组分工配合：讲解、演示、回答问题衔接流畅，无配合失误；
 2. 成员精神状态：答辩过程认真、态度端正，无敷衍情况；
 3. 问题抽查效果：被抽查成员能准确回答基础问题。
- 评分标准：表现优秀得10分；配合混乱、敷衍扣2-5分。

(三) 答辩总结与反馈

1. 每组答辩结束后，讲师针对代码问题、设计缺陷、技术不足给出具体改进建议；
2. 汇总各组得分，评选**最佳实战小组**（综合得分最高），给予荣誉与额外奖励。